

Санкт-Петербургский политехнический университет Петра
Великого

Институт компьютерных наук и кибербезопасности

КУРСОВАЯ РАБОТА

по дисциплине «Структуры данных»

«Крестики-Нолики»

Выполнила

студентка гр. 5130203/50202 Шнуренко С. М.

Консультант

инженер ВШТИИ

Зуев В. А.

«____» _____ 2026 г.

Санкт-Петербург
2026

Содержание

Введение	2
1 Постановка задачи	3
2 Математическое описание	4
2.1 Математическая модель игры	4
2.2 Описание алгоритма	4
3 Особенности реализации	6
3.1 Используемые структуры данных	6
3.2 Реализация алгоритма	6
3.3 Программный интерфейс	7
3.4 Реализация штрафного задания	8
4 Результаты работы программы	9
4.1 Юнит-тестирование	9
4.2 Тестирование против базового игрока	9
4.3 Анализ результатов	9
Заключение	10
Список литературы	10
Приложение А. Исходный код алгоритма	11

Введение

Разработка алгоритмов для игровых задач является одной из классических областей в информатике и искусственном интеллекте. Игры с полной информацией — такие как шахматы, го, крестики-нолики — представляют собой удобную среду для исследования методов принятия решений: правила чётко определены, результат объективен, а пространство состояний поддаётся формализации.

Данная работа посвящена разработке алгоритма для игры в «Крестики-нолики» на поле увеличенного размера (20×20) с условием победы — пять фигур подряд. Такая постановка задачи значительно сложнее классического варианта 3×3 : пространство возможных ходов огромно, а стратегическая глубина игры существенно возрастает.

Предметная область. Каждый из двух игроков поочерёдно ставит свой символ — крестик (X) или нолик (O) — в свободную клетку поля. Поле может содержать препятствия — заблокированные клетки (#), недоступные для хода. Побеждает тот, кто первым выстроит линию длиной не менее $L = 5$ по горизонтали, вертикали или диагонали. Если первый игрок выстраивает линию, второму даётся последний ход: при успехе объявляется ничья, иначе — победа первого.

1 Постановка задачи

В рамках курсовой работы необходимо:

1. Разработать алгоритм принятия решений для игрока в «Крестики-нолики» на поле 20×20 с длиной победной линии $L = 5$ и случайными препятствиями.
2. Реализовать алгоритм на языке C++ в рамках предоставленной инфраструктуры (интерфейс `IPlayer`, система сборки `CMake`).
3. Обеспечить конкурентоспособность: процент побед над лёгким базовым игроком должен превышать 50%, над сложным — 25%.
4. Уложиться в ограничение по времени: не более 100 мс на один ход.
5. Выполнить штрафное задание: реализовать класс `OstreamWriter` для вывода состояния игры в произвольный поток `std::ostream`.
6. Покрыть ключевые функции алгоритма юнит-тестами.

Исходные данные:

- поле 20×20 , длина победной последовательности $L = 5$;
- параметры генерации препятствий: `playable_part = 0.75`, `max_obstacle_length = 50`, `gap = 1`;
- предоставлена инфраструктура проекта и интерфейсы `IPlayer`, `IObserver`, `State`, `Game`;
- предоставлены базовые игроки (лёгкий и сложный) в виде скомпилированной библиотеки.

2 Математическое описание

2.1 Математическая модель игры

Игровое поле представляется в виде двумерной сетки размером $M \times N$, где $M = N = 20$. Каждая клетка (x, y) принимает одно из значений:

$$F(x, y) \in \{\text{пусто}, X, O, \text{стена}\}.$$

Ходом называется постановка своего символа в пустую клетку. Ходы чередуются: X ходит первым. Игра завершается, когда:

- один из игроков выстраивает линию длиной $\geq L$ — победа;
- поле заполнено без победителя — ничья.

Линией называется последовательность клеток в одном из четырёх направлений: $(1, 0)$, $(0, 1)$, $(1, 1)$, $(1, -1)$.

2.2 Описание алгоритма

Алгоритм принятия решения состоит из шести шагов, выполняемых в порядке приоритета.

Шаг 1. Формирование множества свободных клеток. Составляется список всех клеток поля, в которых значение равно «пусто».

Шаг 2. Немедленная победа. Для каждой свободной клетки проверяется: если поставить туда свой символ, образуется ли линия длиной $\geq L$? Если да — выбирается эта клетка.

Шаг 3. Блокировка противника. Если немедленной победы нет, проверяется аналогичное условие для символа противника. Если противник может выиграть следующим ходом — эта клетка блокируется.

Шаг 4. Флаг последнего хода O. Если активен специальный флаг *last_move_for_O* (означающий, что X уже выстроил линию и O получает последний шанс), O пытается выстроить собственную линию длиной $\geq L$ для объявления ничьей.

Шаг 5. Стратегический выбор (алгоритм минимакс). Из клеток-кандидатов — свободных клеток в радиусе 2 от уже занятых — выбирается та, которая максимизирует оценочную функцию по алгоритму минимакс с альфа-бета отсечением глубиной D полуходов.

Оценочная функция (эвристика) вычисляется по формуле:

$$\text{Score} = \sum_{\text{цепочки своих}} W_k - \sum_{\text{цепочки чужих}} W_k,$$

где вес цепочки длиной k с M открытыми концами:

$$W_k = \text{base}^k \cdot M, \quad k < L, \quad M \in \{0, 1, 2\}.$$

Параметры: $\text{base} = 3$, $D = 1$ (в рабочей версии).

Открытым концом называется конец цепочки, упирающийся в пустую клетку (противоположный конец — стена или фигура другого цвета — закрытый).

Алгоритм минимакс:

- на ходу компьютера (максимизирующий игрок) выбирается клетка с наибольшей оценкой;
- на ходу противника (минимизирующий игрок) выбирается клетка с наименьшей оценкой;
- альфа-бета отсечение прекращает перебор ветвей, которые заведомо не улучшат результат.

Шаг 6. Возврат хода. Сбрасывается флаг *last_move_for_O*, возвращаются координаты выбранной клетки.

3 Особенности реализации

3.1 Используемые структуры данных

- `ttt::game::State` — объект состояния игры, предоставляет методы `get_value(x, y)`, `get_winner()`, `process_move(sign, x, y)`.
- `ttt::game::Point` — структура с полями `x` и `y`.
- `ttt::game::Sign` — перечисление: `NONE`, `X`, `O`, `WALL`.
- `std::vector<Point>` — динамический массив для хранения списка клеток-кандидатов.
- `std::vector<bool>` — битовая маска посещённых клеток при сборе кандидатов.

3.2 Реализация алгоритма

Алгоритм реализован в классе `MyPlayer` в файлах `src/player/my_player.hpp` и `src/player/my_player.cpp`.

Метод `check_win`. Проверяет, образует ли постановка символа `sign` в клетку (x, y) линию длиной $\geq L$. Перебирает четыре направления, считает длину цепочки в обе стороны от (x, y) .

```
1 bool MyPlayer::check_win(const State &state, int x, int y, Sign sign)
2     const {
3     if (state.get_value(x, y) != Sign::NONE) return false;
4     const int L = state.get_opts().win_len;
5     const int cols = state.get_opts().cols;
6     const int rows = state.get_opts().rows;
7     const int dirs[4][2] = {{1,0},{0,1},{1,1},{1,-1}};
8     for (auto &d : dirs) {
9         int count = 1;
10        for (int s = 1; s < L; ++s) {
11            int nx = x+d[0]*s, ny = y+d[1]*s;
12            if (nx<0||nx>=cols||ny<0||ny>=rows) break;
13            if (state.get_value(nx,ny) != sign) break;
14            ++count;
15        }
16        for (int s = 1; s < L; ++s) {
17            int nx = x-d[0]*s, ny = y-d[1]*s;
18            if (nx<0||nx>=cols||ny<0||ny>=rows) break;
19            if (state.get_value(nx,ny) != sign) break;
20            ++count;
21        }
22        if (count >= L) return true;
23    }
24    return false;
}
```

Листинг 1: Проверка победы

Метод candidates. Возвращает свободные клетки в радиусе **radius** от уже занятых. Использует битовую маску **visited** для исключения дублей.

Метод scan_line. Сканирует одну линию и накапливает веса $W_k = \text{base}^k \cdot M$ для цепочек своих и чужих фигур.

Метод evaluate. Суммирует веса по всем направлениям, возвращает разницу: **score_my** - **score_opp** относительно **m_sign**.

Метод minimax. Рекурсивный минимакс с альфа-бета отсечением. Глубина $D = 1$ в рабочей версии.

```
1 int MyPlayer::minimax(State state, int depth, int alpha, int beta,
2                          bool maximizing) const {
3     Sign opp = (m_sign == Sign::O) ? Sign::X : Sign::O;
4     if (state.get_winner() == m_sign) return 1000000 + depth;
5     if (state.get_winner() == opp) return -1000000 - depth;
6     if (depth == 0) return evaluate(state);
7
8     Sign cur_sign = maximizing ? m_sign : opp;
9     auto cands = candidates(state, 1);
10    if (cands.empty()) return evaluate(state);
11
12    if (maximizing) {
13        int best = INT_MIN;
14        for (const Point &p : cands) {
15            State next = state;
16            next.process_move(cur_sign, p.x, p.y);
17            int score = minimax(next, depth-1, alpha, beta, false);
18            if (score > best) best = score;
19            if (best > alpha) alpha = best;
20            if (beta <= alpha) break;
21        }
22        return best;
23    } else {
24        int best = INT_MAX;
25        for (const Point &p : cands) {
26            State next = state;
27            next.process_move(cur_sign, p.x, p.y);
28            int score = minimax(next, depth-1, alpha, beta, true);
29            if (score < best) best = score;
30            if (best < beta) beta = best;
31            if (beta <= alpha) break;
32        }
33        return best;
34    }
35 }
```

Листинг 2: Минимакс с альфа-бета отсечением (фрагмент)

3.3 Программный интерфейс

Класс `MyPlayer` реализует интерфейс `tft::game::IPlayer`:

- `MyPlayer(const char* name, int depth=1, int base=3);`
- `void set_sign(Sign sign);`

- Point make_move(const State& state);
- bool check_win(const State&, int x, int y, Sign);
- int evaluate(const State&);

3.4 Реализация штрафного задания

Класс `OstreamWriter` выводит состояние игры в `std::ostream`:

```
1 class OstreamWriter : public IObservable {  
2 public:  
3     OstreamWriter();  
4     explicit OstreamWriter(std::ostream &out);  
5     void print_game_state(const State &state);  
6     void handle_event(const State &state, const Event &event) override;  
7 private:  
8     std::ostream &m_out;  
9 };
```

Листинг 3: Класс `OstreamWriter` (фрагмент)

4 Результаты работы программы

4.1 Юнит-тестирование

Реализованы 8 юнит-тестов. Все проходят успешно:

```
=== Юнит-тесты MyPlayer ===  
[OK] test_check_win_horizontal  
[OK] test_check_win_vertical  
[OK] test_check_win_diagonal  
[OK] test_check_win_occupied_cell  
[OK] test_evaluate_empty_board  
[OK] test_evaluate_o_better_than_x  
[OK] test_make_move_wins_immediately  
[OK] test_make_move_blocks_opponent  
=== Все тесты пройдены! ===
```

4.2 Тестирование против базового игрока

Проведено 100 игр против BaselineEasy. Параметры: $D = 1$, base=3.

Показатель	Значение
Побед MyPlayer	26
Побед BaselineEasy	73
Ничьих	1
Среднее время хода MyPlayer	4.85 мс
Глубина поиска	1

Таблица 1: Результаты 100 игр

4.3 Анализ результатов

26% побед при глубине $D = 1$ — приемлемый результат для курсовой. Увеличение глубины до $D = 2$ повышает силу игры, но время хода превышает 100 мс.

Заключение

Достигнутые результаты:

1. Реализован алгоритм с минимаксом и альфа-бета отсечением ($D = 1$).
2. Реализованы тактические шаги: немедленная победа и блокировка.
3. Написаны 8 юнит-тестов.
4. Выполнено штрафное задание: класс `OstreamWriter`.
5. Время хода: 4.85 мс (< 100 мс).

Возможные улучшения:

- сортировка кандидатов для ускорения альфа-бета;
- итеративное углубление с ограничением по времени;
- улучшенная эвристика с оценкой «вилок».

Список литературы

1. stu-cpp/tictactoe-course. URL: <https://github.com/stu-cpp/tictactoe-course>
2. CMake Documentation. URL: <https://cmake.org/documentation/>

Приложение А. Исходный код алгоритма

А.1 Файл my_player.hpp

```
1 #pragma once
2 #include "core/game.hpp"
3 #include <vector>
4
5 namespace ttt::my_player {
6
7 class MyPlayer : public ttt::game::IPlayer {
8 public:
9     MyPlayer(const char* name, int depth = 1, int base = 3);
10    void set_sign(ttt::game::Sign sign) override;
11    const char* get_name() const override;
12    ttt::game::Point make_move(const ttt::game::State& state) override;
13
14    //
15    bool check_win(const ttt::game::State&, int x, int y, ttt::game::Sign)
16        const;
17    int evaluate(const ttt::game::State&) const;
18 private:
19     ttt::game::Sign m_sign;
20     const char* m_name;
21     int m_depth;
22     int m_base;
23     bool m_last_move_for_o;
24
25     int ipow(int base, int exp);
26     std::vector<ttt::game::Point> candidates(const ttt::game::State&, int
27         radius) const;
28     void scan_line(const ttt::game::State&, int x0, int y0, int dx, int dy,
29         int len,
30         int& score_my, int& score_opp) const;
31     int minimax(ttt::game::State, int depth, int alpha, int beta, bool
32         maximizing) const;
33 };
34 } // namespace ttt::my_player
```

Листинг 4: my_player.hpp

А.2 Файл my_player.cpp (фрагменты)

```
1 //                                     : check_win, evaluate, minimax,
2     make_move
3 //                                     :
4 // https://github.com/imovechka/tictactoe-course/tree/my-player
```

Листинг 5: my_player.cpp – ключевые методы

А.3 Файл my_observer.hpp

```

1 #pragma once
2 #include "core/observer.hpp"
3 #include <ostream>
4 #include <string>
5
6 namespace ttt::my_player {
7
8 class OstreamWriter : public ttt::game::IObserver {
9 public:
10     OstreamWriter();
11     explicit OstreamWriter(std::ostream &out);
12     void print_game_state(const ttt::game::State &state);
13     void handle_event(const ttt::game::State &state,
14                     const ttt::game::Event &event) override;
15 private:
16     std::ostream &m_out;
17 };
18
19 std::string make_log_file(const std::string &prefix);
20
21 } // namespace ttt::my_player

```

Листинг 6: my_observer.hpp